

GIORNO 01

Cos'è un linguaggio di programmazione: come si dice a un computer cosa deve fare

Cosa saprete fare stasera

Scrivere un file .py, salvarlo nella cartella giusta ed eseguirlo dal terminale

Spiegare, con due conseguenze pratiche, perché Python è interpretato

Comporre un output su più righe con print(): virgola, sep, end, escape

Dare un nome a un valore e riutilizzarlo in più punti senza riscriverlo

Leggere un traceback, dire dove è saltato il programma e correggerlo

Copiare la struttura standard di uno script: docstring, costanti, main, guard

Indice della giornata (1/2)

1. Benvenuti: come funziona questo corso
2. Cosa succede quando un computer esegue qualcosa
3. Cos'è un linguaggio di programmazione
4. Compilati, interpretati e vie di mezzo
5. Perché proprio Python
6. Installare l'ambiente e verificarlo
7. Il terminale senza paura
8. Il REPL: la palestra

Indice della giornata (2/2)

- 9. Il primo programma su file
- 10. `print()`: l'unica finestra sul mondo
- 11. Dare un nome a un valore
- 12. Numeri e testo al primo sguardo
- 13. Commenti e leggibilità
- 14. Errori e traceback: leggerli senza panico
- 15. La struttura standard di uno script
- 16. Come si legge un programma scritto da altri

Si chiude con il riepilogo e la mappa degli otto esercizi.

1. BENVENUTI: COME FUNZIONA QUESTO CORSO

L'intero corso, dall'inizio alla fine (1/2)

Fase del percorso	Che cosa ci si mette in mano
Le fondamenta di Python	Istruzioni, variabili e tipi di dato, input e output, decisioni
L'intelligenza artificiale, i concetti	Cos'è davvero l'AI oggi; AI, machine learning e deep learning; classificare e raggruppare
Competenze, CV e colloquio	Analisi delle proprie competenze, curriculum, colloquio, comunicazione e lavoro in team
Ripetere, organizzare, conservare	Cicli, funzioni, liste, dizionari e tuple, lettura e scrittura di file
I dati in tabella	Modello Entità-Relazione, progettazione delle tabelle, normalizzazione, chiavi, SQL

Duecentoquarantotto ore in due fasi dichiarate: prima Python, per mettere basi che reggano; poi l'AI applicata con Python. In partenza non serve niente: il percorso è pensato per chi non ha mai scritto una riga di codice.

L'intero corso, dall'inizio alla fine (2/2)

Fase del percorso	Che cosa ci si mette in mano
Python nel lavoro vero	Cosa sono i Large Language Models, gestione degli errori, script realistici, NumPy
AI applicata con Python	API AI e prompt engineering, Pandas, file CSV ed Excel, RAG, chatbot, AI agent, automazioni
I progetti di chiusura	Mini-progetto Python, progetto finale guidato, preparazione al colloquio tecnico
Moduli obbligatori	Salute e sicurezza sui luoghi di lavoro; diritti e doveri dei lavoratori

Tre fasi di questa mappa sono le undici giornate che cominciano oggi. Tutto il resto arriva più avanti nel corso, e poggia su quello che facciamo qui: chi non sa cos'è una variabile non può usare una libreria di analisi dati.

2. COSA SUCCEDE QUANDO UN COMPUTER ESEGUE QUALCOSA

La macchina più ubbidiente e più stupida del mondo

Il computer non capisce niente

Quando il programma non fa quello che volete, il primo pensiero è:

«il computer non ha capito».

Sbagliato: ha capito benissimo e ha fatto esattamente quello che avete scritto.
Quello che avete scritto non è quello che intendevate.

Analogia — il montaggio del mobile in scatola

il libretto di istruzioni è il programma, voi siete il processore

«inserire la vite A nel foro B», e il foro B non esiste: vi fermate
non inventate un foro alternativo

Quel fermarsi, il computer lo chiama errore.

I tre pezzi che vi serve conoscere

Componente	Cosa fa	L'analogia in cucina
Processore (CPU)	Esegue una istruzione alla volta, molto in fretta	Il cuoco: due mani, una operazione per volta
Memoria (RAM)	Tiene i dati mentre il programma gira	Il piano di lavoro: ci sta poco, ma ci arrivi subito
Disco (SSD)	Conserva i file anche a computer spento	La dispensa: ci sta tanto, ma devi andarci

Quando il programma finisce, la memoria si svuota: ogni lancio riparte da zero. Per questo il gestionale di NovaStore scrive l'ordine su disco appena lo confermate, e ai file arriviamo nell'ultima giornata di questo blocco.

Due cose da portarsi dietro

Il processore esegue in ordine, dall'alto verso il basso.

riga 1, poi riga 2, poi riga 3: non salta avanti a leggere cosa viene dopo
usare un valore prima di averlo creato ferma il programma: capitolo 14

Dentro al processore esistono solo sequenze di zero e uno.

non esistono lettere, non esistono euro, non esistono i cartellini
la lettera A, per il computer, è il numero 65

il 7 scritto fra virgolette e il 7 con cui si somma sono due cose diverse
con il primo la somma non si fa: ci sbattiamo contro al capitolo 12

Non serve impararlo. Serve non stupirsi quando succede.

Dal vostro file all'esecuzione

primo.py

file di testo
scritto da voi

->

python

legge il file
riga per riga

->

processore

esegue le
istruzioni

Il pezzo di mezzo è un programma installato sul vostro computer e si chiama Python: come Word, solo che invece di documenti apre file di istruzioni.

3. COS'È UN LINGUAGGIO DI PROGRAMMAZIONE

Precisione al posto dell'ambiguità

«Fammi un caffè» non è un programma

Il primo tentativo di tutti:

1. Riempi la caldaia d'acqua 2. Metti il caffè 3. Avvita 4. Sul fuoco

Chi la moka non l'ha mai vista chiede:

quanta acqua? fino all'orlo? il caffè dove, dentro l'acqua?

avvita cosa a cosa? quale fuoco, e per quanto tempo?

Analogia — la ricetta di cucina

«fai una torta» non è un programma; «accendi il forno a 180 gradi» lo è

la differenza non è la lunghezza: è la precisione

Un linguaggio non serve a parlare con il computer:

serve a costringere voi a essere precisi.

Le quattro parole che sentirete ripetere

Parola	Cosa vuol dire	Se la sbagliate
Sintassi	Le regole di scrittura: parentesi, virgolette, due punti	Il programma non parte proprio
Semantica	Il significato di quello che avete scritto	Il programma parte e fa la cosa sbagliata
Astrazione	Usare uno strumento senza sapere come è fatto dentro	Niente: è il motivo per cui si programma
Precisione	Ogni istruzione ha un effetto e uno solo	È il patto: se è preciso, è ripetibile

`prnt("Ciao")` è sintassi: Python non conosce la parola e si ferma subito. `print("Ciao")` quando volevate il nome del cliente è semantica: parte e sbaglia, e non c'è nessun messaggio d'errore — solo un cartellino sbagliato sullo scaffale.

Perché non si programma in italiano

In italiano	Quante letture ha
«Aggiungi lo sconto al prezzo»	Lo sommo? Lo tolgo? È in euro o in percentuale?
«Metti in ordine i clienti»	Per nome? Per data? Crescente o decrescente?
«Prendi l'ultimo ordine»	L'ultimo inserito o l'ultimo in ordine di data?
«Stampa»	A schermo o sulla stampante?

Le parole riservate di Python sono poco più di trenta e in tutto il corso ne useremo una decina: la difficoltà non è il vocabolario, è spezzare un problema in passi abbastanza piccoli da poterli scrivere.

4. COMPILATI, INTERPRETATI E VIE DI MEZZO

Perché con Python si prova in due secondi

Due modi di tradurre

Il file che scrivete è testo. Il processore mangia solo numeri.
Qualcuno deve tradurre, e ci sono due scuole.

Analogia — il traduttore simultaneo contro il libro tradotto

il compilatore prende tutto il libro, se lo porta a casa, lo traduce
da lì la lettura è velocissima, ma per cambiare una virgola si ristampa
l'interprete è il traduttore seduto accanto a voi: traduce mentre parlate
più lento, ma potete correggervi al volo e ricominciare da dove eravate

Python è il traduttore simultaneo.

Cosa cambia davvero, nella giornata di lavoro

	Compilato (C, Rust, Go)	Interpretato (Python, JavaScript)
Prima di provare	Serve un passaggio di compilazione	Si lancia e basta
Velocità di esecuzione	Più alta	Più bassa
Ciclo di lavoro	Scrivi, compila, esegui, ripeti	Scrivi, esegui, ripeti
L'errore di battitura salta fuori	Alla compilazione, prima di eseguire	Quando l'esecuzione arriva su quella riga
Correzione di un dettaglio	Ricompilare tutto	Salvare e rilanciare

Le due righe che contano per voi sono le ultime due.

La brutta notizia, la buona notizia e il bytecode

L'errore di battitura salta fuori tardi.

un nome scritto male alla riga 200 non impedisce al programma di partire
esegue le prime 199 righe e si schianta quando ci arriva
è il prezzo della comodità: si paga leggendo i messaggi d'errore

Il giro veloce è la ragione per cui oggi imparate in fretta.

fra lo scrivere una riga e il vederla funzionare passano due secondi
scrivo, provo, sbaglio, correggo: è il vero motore dell'apprendimento

Python, prima di eseguire, traduce il file in una forma intermedia.

si chiama bytecode e finisce nella cartella `__pycache__`
non è un virus, non si cancella a mano: se la cancellate, la ricrea

5. PERCHÉ PROPRIO PYTHON

Si legge, c'è già tutto pronto, ed è lento

Si legge: un programma di tre righe

```
print("Poliambulatorio Aurora")  
print("Accettazione")  
print("Sportello 3")
```

print significa stampa, le parentesi contengono quello che va stampato.

Dove si usa davvero

Ambito	Esempio concreto
Automazione d'ufficio	Rinominare 4000 file, unire venti report in uno
Analisi dati	Leggere estrazioni gestionali e produrre statistiche
Intelligenza artificiale	Quasi tutti gli strumenti del settore parlano Python
Siti e servizi web	La parte che sta sul server, non quella che vedete
Scienza	Fisica, biologia, statistica, elaborazione di immagini

Le due cose che non vanno

Un docente che vi elenca solo i pregi vi sta vendendo qualcosa.

È lento.

molto più lento di un compilato sullo stesso calcolo

per quasi tutto quello che farete è irrilevante: mezzo secondo al giorno

diventa rilevante sui milioni di righe, e lì si usano le librerie giuste

L'ambiente si rompe.

il codice Python funziona quasi sempre; è il contorno a far perdere le ore

versioni diverse sulla stessa macchina, pacchetti che litigano

il comando è python su un computer e python3 su un altro

Metà della fatica del prossimo capitolo nasce da qui, e non è colpa vostra.

6. INSTALLARE L'AMBIENTE E VERIFICARLO

Due cose da installare, tre comandi per verificarle

Cosa installiamo, e nient'altro

Cosa	Versione	A cosa serve
Python	3.12 (va bene anche 3.13)	Il programma che esegue i vostri programmi
PyCharm	l'ultima	L'editor in cui scrivete

Il supporto Python è già dentro PyCharm: nessuna estensione da aggiungere. Su Windows, durante l'installazione di Python, spuntate «Add python.exe to PATH»: è la singola casella più importante della giornata.

Scaricare PyCharm: quale file

<https://www.jetbrains.com/pycharm/download/>

Windows	installer .exe, oppure archivio .zip
macOS con processore Intel	il .dmg per Intel
macOS con Apple Silicon	il .dmg per Apple Silicon
Linux	archivio .tar.gz

Le due righe del Mac sono la trappola: i due .dmg si chiamano quasi uguale e stanno uno accanto all'altro. Senza diritti di amministratore: l'installer .exe accetta una cartella dentro la vostra home, oppure lo .zip si estrae e si avvia.

Trenta giorni, e il computer che non ce la fa

Il conto alla rovescia che vedrete comparire.

il nucleo di PyCharm è gratuito e non scade: con quello si fa tutto il corso
ogni installazione nuova parte con 30 giorni di prova delle funzioni Pro
arriveranno a zero mentre il corso è ancora aperto, e non si blocca niente
si spengono le funzioni in prova, il resto lavora come il primo giorno

Se la macchina è vecchia.

la documentazione chiede 8 GB di RAM (3 liberi per l'editor) e ~10 GB di disco
su un portatile di sette o otto anni fa può diventare pesante sul serio

In quel caso: IDLE, che arriva con Python e non va installato.

apre i file, li salva, li esegue: spartano, ma basta
non è la scelta di serie B: senza librerie né import gira tutto uguale
se siete in questa situazione ditelo subito, non a fine giornata

Il protocollo di verifica

Windows (Prompt dei comandi o PowerShell)

```
python --version
```

macOS o Linux (Terminale)

```
python3 --version
```

La risposta deve essere una riga come questa:

```
Python 3.12.4
```

Poi, per sapere che l'installazione è completa:

```
pip --version
```

Deve cominciare con 3.12 o 3.13. Se leggete Python 2.7 è la vecchia versione di sistema del Mac. Scrivete in chat la riga esatta che vi ha risposto il comando: cinque minuti, ed è il censimento più utile della giornata.

I tre problemi che avremo di sicuro

Sintomo	Causa	Via d'uscita
'python' non è riconosciuto come comando (Windows)	Installato senza spuntare «Add to PATH»	Rilanciare l'installer, Modify, spuntare la casella, riaprire il terminale
python risponde Python 2.7.x (macOS)	Sul Mac c'è una vecchia versione di sistema	Usare sempre python3 al posto di python, in ogni comando
L'installer si blocca o sparisce (PC aziendale)	Antivirus o criteri aziendali	Installer «user level», solo per l'utente corrente, senza diritti di admin

Il secondo caso non è un problema da risolvere: è una differenza da ricordare. Nei materiali lo trovate scritto così: python (o python3).

La cartella di lavoro

```
Documenti/  
  corso-python/  
    giorno_01/  
    giorno_02/  
    ...
```

Niente spazi (corso-python sì, corso python no), niente accenti, tutto minuscolo. In PyCharm si apre la cartella, non il singolo file: File > Open. Diventa il progetto, e PyCharm chiede quale Python usare: indicate il 3.12 appena installato.

7. IL TERMINALE SENZA PAURA

Quattro comandi, e l'errore che faremo tutti

I quattro comandi

Cosa volete fare	Windows	macOS / Linux
Vedere dove siete	cd	pwd
Vedere cosa c'è qui	dir	ls
Entrare in una cartella	cd giorno_01	cd giorno_01
Tornare indietro di una	cd ..	cd ..

Il tasto TAB completa i nomi, e vi dice pure se la cartella esiste: se non completa niente, quella cartella lì non c'è. La freccia in su ripescava l'ultimo comando.

Lanciare uno script

Windows

```
python primo.py
```

macOS

```
python3 primo.py
```

Si legge così: «Python, esegui il file primo.py che sta nella cartella in cui mi trovo adesso». Oggi si lancia dal terminale e non dal triangolino verde di PyCharm: il terminale vi lascia l'errore davanti, e sul PC di lavoro c'è sempre.

L'errore numero uno del primo giorno

```
python: can't open file 'primo.py': [Errno 2] No such file or directory
```

Il file esiste, ma non è nella cartella in cui è aperto il terminale:

```
il file sta in      Documenti/corso-python/giorno_01/  
il terminale sta in Documenti/
```

Rimedio sempre uguale: dir (o ls) per vedere cosa c'è dove siete. Se il vostro file non compare nella lista, siete nel posto sbagliato: cd fino ad arrivarci, poi rilanciate.

8. IL REPL: LA PALESTRA

Provare una riga alla volta

Aprire il REPL

```
$ python
Python 3.12.4 (main, Jun 6 2024, 18:26:44)
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> "Nova" + "Store"
'NovaStore'
>>> print("Ciao")
Ciao
>>> exit()
```

2 + 3 ha risposto 5 anche senza print: nel REPL il risultato di ogni riga si vede da solo. Dentro un file questo non succede: se non scrivete print, non vedete niente. È la prima sorpresa di tutti.

Quando conviene e quando no

Il REPL è comodo per	Il REPL è pessimo per
Verificare cosa fa un'istruzione	Scrivere un programma di venti righe
Fare un calcolo al volo	Riprendere il lavoro di ieri
Provare un dubbio nato in aula	Consegnare qualcosa a qualcuno

Il motivo è uno solo: il REPL non salva niente. È una lavagna, non un quaderno. Regola pratica: il REPL per i dubbi da trenta secondi, il file per tutto il resto.

9. IL PRIMO PROGRAMMA SU FILE

Quattro gesti, sempre gli stessi, per undici giornate

I quattro gesti

1. Creare il file, con estensione .py
2. Scrivere dentro le istruzioni
3. Salvare — Ctrl+S, oppure Cmd+S su Mac
4. Eseguire dal terminale

Il terzo passo è quello che si dimentica.

sintomo tipico: correggete l'errore, rilanciate, e l'errore è identico

non è il computer che si è impuntato: sul disco c'è ancora il file vecchio

PyCharm però salva da solo, e questo vi capiterà di rado.

salva quando passate a un'altra applicazione: andare sul terminale è questo

salva dopo qualche secondo di tastiera ferma, e comunque prima di eseguire

Ctrl+S continua a funzionare e forza il salvataggio: prendete l'abitudine

altrove il salvataggio automatico non c'è, e il sintomo va riconosciuto

primo.py

```
print("Poliambulatorio Aurora")  
  
print("Accettazione - Sportello 3")  
  
print("Prossimo turno: 14")
```

Si crea con il tasto destro sulla cartella nel pannello di progetto: New > Python File, poi il nome senza estensione. Il file deve chiamarsi primo.py e non primo.py.txt: per questo niente Blocco Note, che il .txt lo aggiunge di nascosto.

Lancio, risposta e percorso completo

```
$ python primo.py
Poliambulatorio Aurora
Accettazione - Sportello 3
Prossimo turno: 14
```

```
Voi scrivete primo.py -> salvate -> il file sta sul disco
python primo.py       -> Python legge la riga 1, la esegue
                        -> legge la riga 2, la esegue
                        -> legge la riga 3, la esegue
                        -> finisce e restituisce il controllo al terminale
```

Quando il terminale vi ripresenta il prompt, il programma è finito. Il file si può chiamare come volete, purché finisca in .py: un nome da evitare è python.py.

10. `print()`: L'UNICA FINESTRA SUL MONDO

Se non lo stampate, è come se non fosse mai successo

La forma base, e i tre modi di sbagliarla

```
print("Ciao")
```

Tre pezzi: il nome print, le parentesi tonde, dentro quello da stampare.
Il testo va fra virgolette.

Tre modi diversi di fermare il programma, tutti al capitolo 14:

`Print("Ciao")` — con la maiuscola: Print non esiste

`print "Ciao"` — senza parentesi: si scriveva così quindici anni fa

`print(Ciao)` — senza virgolette: Python cerca un nome di variabile

Oggi print è l'unico modo che avete per sapere cosa succede dentro il programma: vale la pena impararlo bene.

Più cose in una volta sola

```
print("Zona:", "NORD")  
print("Colli:", 3)  
print("Peso:", 12.5, "kg")
```

```
Zona: NORD  
Colli: 3  
Peso: 12.5 kg
```

Il 3 non è fra virgolette e print lo stampa lo stesso. Fra un pezzo e l'altro print mette uno spazio da solo, senza che glielo abbiate chiesto.

Il pattern più importante della giornata

```
totale = 450.79
```

```
print("Totale ordine:", totale)
```

Stampa: Totale ordine: 450.79 — con il + il programma salta, e si vede al cap. 12.

sep: cosa mettere in mezzo

```
print("LS", "2026", "MI", "0042", sep="-")    ->    LS-2026-MI-0042
```

```
print("Prezzo", 89.0, sep="")                ->    Prezzo89.0
```

```
print("A", "B", "C", sep=" | ")              ->    A | B | C
```

```
print("riga1", "riga2", sep="\n")            ->    riga1  
                                                riga2
```

sep dice cosa infilare fra un pezzo e l'altro: il valore normale è uno spazio, ma potete metterci quello che volete, compreso niente. Con `sep="\n"` un solo print produce più righe: è un trucco che userete.

end: cosa mettere alla fine, e la riga vuota

```
print("Caricamento", end=" ")    -> Caricamento completato  
print("completato")              (due print, una riga sola)
```

```
print("[", end="")               -> [OK]  
print("OK", end="")  
print("]")
```

```
print("Sopra")                  -> Sopra  
print()                         (riga vuota)  
print("Sotto")                  Sotto
```

Ogni print finisce andando a capo non per legge di natura, ma perché il suo valore normale di fine riga è «a capo»: end lo cambia. Serve quando costruite una riga a pezzi.

I caratteri di escape

Scrivete	Ottenete	Nome
<code>\n</code>	un a capo	newline
<code>\t</code>	una tabulazione	tab
<code>\\</code>	un backslash	backslash
<code>\"</code>	una virgoletta doppia	virgoletta protetta

Il backslash dice a Python: il carattere che segue prendilo alla lettera. Regola pratica sulle virgolette: apostrofo dentro il testo? doppie fuori. Virgolette dentro? apici singoli fuori, oppure protegetele con \".

Escape e tabulazioni in pratica

<code>print("Prima riga\nSeconda riga")</code>	-> Prima riga Seconda riga
<code>print("Nome:\tGiulia")</code>	-> Nome: Giulia
<code>print("Cartella: C:\\Utenti\\giulia")</code>	-> Cartella: C:\Utenti\giulia
<code>print("Il paziente ha detto \"va bene\"")</code>	-> Il paziente ha detto "va bene"
<code>print("Cuffie\t49.90")</code>	-> Cuffie 49.90
<code>print("Hub USB-C\t27.00")</code>	-> Hub USB-C 27.00
<code>print("Monitor 27 pollici\t219.00")</code>	-> Monitor 27 pollici 219.00

Su Windows ogni backslash dei percorsi va raddoppiato. Il tab non sposta di un numero fisso di spazi: salta alla prossima tacca, ogni otto caratteri — appena una parola è lunga, la colonna sballa. Con il tab si convive: l'allineamento serio si fa domani, con strumenti fatti apposta.

Da provare subito

```
print("A", "B", sep="", end="-")  
  
print("C", "D", sep="+")  
  
print("Fine\n")
```

Prima di lanciarle, dite cosa vi aspettate. Output: AB-C+D, poi Fine, poi riga vuota.

11. DARE UN NOME A UN VALORE

Il prezzo scritto in un posto solo

Prima e dopo: lo stesso cartellino NovaStore

Senza variabili

```
print("Cuffie Bluetooth")
print("Prezzo:", 49.9)
print("Cuffie Bluetooth in offerta a 49.9 euro")
print("Scaffale A3 - Cuffie Bluetooth")
```

Con le variabili

```
nome_prodotto = "Cuffie Bluetooth"
prezzo = 49.9

print(nome_prodotto)
print("Prezzo:", prezzo)
print(nome_prodotto, "in offerta a", prezzo, "euro")
print("Scaffale A3 -", nome_prodotto)
```

Sopra il nome è scritto tre volte e il prezzo due: se domani il prezzo cambia, dimenticarne uno è questione di tempo, non di attenzione. Sotto il prezzo sta in un posto solo: si cambia lì e cambia dappertutto.

Che cos'è una variabile

nome_variabile = valore

Il segno = non vuol dire «è uguale a».

vuol dire: da adesso questo nome indica questo valore

si legge da destra a sinistra: prendi il valore, attaccagli l'etichetta

Analogia — il numero in rubrica

salvate il numero e da quel momento chiamate «Mamma», non «339...»

se cambia numero, cambiate la rubrica in un posto solo,

non tutti i bigliettini sparsi per casa

Una variabile si usa solo dopo la riga in cui è stata creata:

Python legge dall'alto in basso, prima è NameError (capitolo 14)

I tre tipi di valore di oggi

```
citta = "Battipaglia"
```

```
numero_colli = 3
```

```
peso_kg = 12.5
```

```
print(citta)
```

```
print("citta")
```

Stampa Battipaglia, poi la parola citta: il nome non va mai fra virgolette.

Le regole per i nomi

Regola	Va bene	Non va bene
Inizia con una lettera o _	zona, _temp	2zona
Solo lettere, cifre e _	peso_kg, zona2	peso-kg, peso kg
Niente parole riservate	stampa_totale	for è vietato, print è pessimo
Maiuscole e minuscole contano	zona e Zona sono due variabili diverse	—

Le altre sono buona educazione, e in questo corso valgono come regole: snake_case (numero_colli, non numeroColli), nomi in italiano, niente abbreviazioni (quantità e non qta). La tastiera completa da sola, il cervello di chi rilegge no.

Nomi cattivi e nomi buoni

Cattivo	Buono	Perché
x	peso_kg	x non dice niente, e nemmeno l'unità di misura
dato1, dato2	nome_paziente, codice_fiscale	I numeri progressivi non spiegano
tmp	totale_parziale	«Temporaneo» resta lì per anni
p	prezzo_unitario	Una lettera è un enigma
prezzoConIva	prezzo_con_iva	Stile sbagliato per Python

Il criterio è uno solo: un buon nome permette di leggere la riga ad alta voce e capirla senza risalire al punto in cui la variabile è stata creata.

12. NUMERI E TESTO AL PRIMO SGUARDO

I quattro operatori, e le due sorprese della divisione

I quattro operatori

```
print(10 + 3)    -> 13
print(10 - 3)    -> 7
print(10 * 3)    -> 30
print(10 / 3)    -> 3.3333333333333335

print(10 / 2)    -> 5.0
print(8 / 4)     -> 2.0
```

Prima sorpresa: i decimali sono tanti, perché il computer non conosce i periodici. Seconda sorpresa, più importante: la divisione produce sempre un numero con la virgola, anche quando il conto viene esatto. 5.0 e 5 sono due cose diverse.

Le operazioni si combinano

```
prezzo_tastiera = 89.00  
prezzo_hub = 27.00  
prezzo_mouse = 34.50  
  
subtotale = prezzo_tastiera + prezzo_hub + prezzo_mouse  
  
print("Subtotale:", subtotale)
```

Stampa 150.5, non 150.50: a Python gli zeri finali non interessano. Si sistema domani.

L'ordine dei conti

Espressione	Risultato	Perché
$2 + 3 * 4$	14	Prima $3 * 4$, poi $2 + 12$
$(2 + 3) * 4$	20	Le parentesi forzano la somma per prima
$100 - 20 / 2$	90.0	Prima $20 / 2$, poi $100 - 10.0$
$(100 - 20) / 2$	40.0	Prima la sottrazione

Le regole sono quelle della scuola. Consiglio valido per tutto il corso: mettete le parentesi anche quando non servono — non cambiano il risultato e tolgono ogni dubbio a chi rilegge, compresi voi fra un mese. Una percentuale si scrive 0.22, non 22%.

Il + fra due testi

```
print("Nova" + "Store")      -> NovaStore
print("LS" + "-" + "2026")   -> LS-2026
```

```
nome = "Giulia"
cognome = "Ferrante"
print(nome + " " + cognome)  -> Giulia Ferrante
print(nome + cognome)        -> GiuliaFerrante
```

```
print("5" + "3")            -> 53
print(5 + 3)                 -> 8
```

Fra due testi il + attacca e basta: lo spazio, se lo volete, lo scrivete voi. Per Python "5" e 5 non sono la stessa cosa, ed è la porta d'ingresso della giornata di domani.

Nota: i decimali si scrivono col punto — 12,5 non è un numero.

L'errore che dobbiamo fare adesso

```
totale = 244.0
print("Totale: " + totale)
```

```
Traceback (most recent call last):
```

```
File "/Users/giulia/corso-python/giorno_01/cassa.py", line 2, in <module>
```

```
    print("Totale: " + totale)
```

```
~~~~~^~~~~~
```

```
TypeError: can only concatenate str (not "float") to str
```

Tradotto: posso attaccare del testo solo ad altro testo, e tu mi hai dato un numero. La soluzione di oggi è la virgola: `print("Totale:", totale)`. Esiste un modo più comodo, la f-string, e arriva domani: oggi si usa la virgola, e si usa apposta.

13. COMMENTI E LEGGIBILITÀ

Scrivere per chi rileggerà, cioè per voi fra un mese

Il cancelletto

```
# Prezzi di listino NovaStore, aggiornati a settembre 2026  
prezzo_tastiera = 89.00  
  
prezzo_hub = 27.00  # promozione di apertura, torna a 32.00 a novembre  
  
# Commento inutile: dice quello che si legge da solo  
prezzo_mouse = 34.50  # assegna 34.50 a prezzo_mouse  
  
# Commento utile: dice una cosa che il codice non dice  
# Prezzi IVA esclusa: il gestionale ce li manda sempre al netto  
prezzo_monitor = 219.00
```

Tutto quello che segue un # viene ignorato da Python: serve alle persone.

PEP 8 in pillole

Regola	Sì	No
Spazi attorno all'= e agli operatori	<code>totale = a + b</code>	<code>totale=a+b</code>
Nessuno spazio dopo la parentesi aperta	<code>print("Ciao")</code>	<code>print("Ciao")</code>
Nomi in snake_case	<code>numero_colli</code>	<code>numeroColli</code>
Costanti in MAIUSCOLO	<code>ALIQUOTA_IVA</code>	<code>aliquotalva</code>
Righe corte	fino a 88 caratteri	righe che scorrono in orizzontale
Indentazione a 4 spazi	4 spazi	tabulazioni, o 2 spazi, o 7
Una riga vuota fra i blocchi logici	—	venti righe attaccate

In Python gli spazi a inizio riga hanno un significato: non sono decorazione. Uno spazio di troppo davanti a una riga blocca il programma, e lo vediamo subito.

Docstring e costanti: lo sportello del Comune di Villanova

```
"""Preventivo per una pratica allo sportello del Comune di Villanova."""
```

```
MARCA_DA_BOLLO = 16.00
```

```
DIRITTI_SEGRETERIA = 25.50
```

```
totale = MARCA_DA_BOLLO + DIRITTI_SEGRETERIA
```

```
print("Totale da versare:", totale)
```

Stampa: Totale da versare: 41.5. Un numero senza nome dentro un calcolo è un magic number.

14. ERRORI E TRACEBACK: LEGGERLI SENZA PANICO

Il messaggio d'errore è dalla vostra parte

Anatomia del messaggio

```
Traceback (most recent call last):
```

```
File "/Users/giulia/corso-python/giorno_01/badge.py", line 7, in <module>
```

```
    print("Totale: " + totale)
```

```
~~~~~^~~~~~
```

```
TypeError: can only concatenate str (not "float") to str
```

```
riga File ..... in quale file e a quale riga: DOVE
```

```
ultima riga .... il tipo di errore e la spiegazione: COSA
```

Analogia: il traceback è lo scontrino dell'errore. Si legge dal basso — l'ultima riga dice cosa è successo, come il totale in fondo; le righe sopra dicono dove, come la lista della spesa. Tutto il resto è contorno.

SyntaxError: non parte proprio

bolla.py

```
1 print("LogiSud Trasporti")
2 print("Bolla di spedizione"
3 print("Zona: NORD")
```

```
File "/Users/giulia/corso-python/giorno_01/bolla.py", line 2
```

```
    print("Bolla di spedizione"
```

```
        ^
```

```
SyntaxError: '(' was never closed
```

La riga 1 non è stata eseguita: con un errore di sintassi non viene stampato niente, nemmeno quello che veniva prima. Se la riga indicata vi sembra perfetta, guardate quella sopra: nove volte su dieci la parentesi mancante è lì.

NameError: non so chi sia

```
File "/Users/giulia/corso-python/giorno_01/badge.py", line 4, in <module>
    print("Totale:", totale)
          ^^^^^
```

NameError: name 'totale' is not defined

```
File "/Users/giulia/corso-python/giorno_01/badge.py", line 1, in <module>
    Print("Ciao")
    ^^^^^
```

NameError: name 'Print' is not defined

Tre cause, in ordine di frequenza: errore di battitura; maiuscole (Totale e totale sono due nomi diversi); ordine delle righe, cioè variabile creata dopo il punto in cui la usate. Print con la maiuscola non esiste: esiste print, tutto minuscolo.

IndentationError: c'è uno spazio di troppo

```
File "/Users/giulia/corso-python/giorno_01/badge.py", line 2
    print("Seconda riga")
```

IndentationError: unexpected indent

```
File "/Users/giulia/corso-python/giorno_01/badge.py", line 4
    print("Ciao")
    ^
```

IndentationError: expected an indented block after function definition on line 3

Il primo dice: questa riga comincia più a destra delle altre e non so perché. Il secondo è il contrario: dopo def main(): Python si aspetta righe rientrate di quattro spazi. Regola di oggi: tutto al margine, tranne il blocco di main().

Tabella di pronto soccorso

Messaggio	Cosa è successo	Prima cosa da guardare
SyntaxError: '(' was never closed	Parentesi non chiusa	La riga indicata e quella sopra
SyntaxError: unterminated string literal	Virgolette non chiuse	Inizio e fine del testo
SyntaxError: invalid syntax	Qualcosa che Python non riconosce	Due punti, virgole, parole storpiate
NameError: name 'x' is not defined	Nome mai creato	Battitura, maiuscole, ordine delle righe
TypeError: can only concatenate str...	+ fra testo e numero	Sostituire il + con la virgola
IndentationError: unexpected indent	Spazi di troppo a inizio riga	Il margine sinistro della riga
IndentationError: expected an indented block	Manca il rientro dopo i due punti	Le righe dentro def main():
ZeroDivisionError: division by zero	Diviso per zero	Il valore del divisore

Il metodo, in cinque passi

1. Non toccare niente. Leggete.
2. Ultima riga: che tipo di errore è.
3. Riga con File: quale file, quale numero di riga.
4. Aprite quella riga, e guardate anche quella sopra.
5. Una modifica alla volta, poi rilanciate.

Cambiare tre cose insieme e veder sparire l'errore non insegna niente:
non sapete quale delle tre ha funzionato.

Python segnala il primo errore e si ferma: quando lo correggete
può comparirne un altro. Non avete peggiorato le cose, state avanzando.

15. LA STRUTTURA STANDARD DI UNO SCRIPT

Una formula da copiare: si spiega quando arriveranno le funzioni

La formula

```
"""Cartellino prezzo per il magazzino NovaStore."""
```

```
PREZZO_TASTIERA = 89.00
```

```
SCAFFALE = "A3"
```

```
RIGA = "=====
```

```
def main():
```

```
    """Stampa il cartellino di un articolo."""
```

```
    nome_prodotto = "Tastiera meccanica"
```

```
    print(RIGA)
```

```
    print(nome_prodotto)
```

```
    print("Prezzo:", PREZZO_TASTIERA)
```

```
    print("Scaffale:", SCAFFALE)
```

```
    print(RIGA)
```

```
if __name__ == "__main__":
```

```
    main()
```

RIGA è il modo di oggi per i separatori: si scrive una volta, si stampa dove serve.

Le cinque parti, e le tre cose da non sbagliare

Docstring — a cosa serve il file, in due o tre righe

Costanti — i valori fissi, in MAIUSCOLO

def main(): — il programma vero, rientrato di quattro spazi

Due righe vuote — prima del def e prima del guard, per convenzione

Guard finale — le due righe che fanno partire tutto

Le tre cose da non sbagliare:

- i due punti in fondo a def main(): — se mancano, `SyntaxError: expected ':'`

- il rientro: una riga incollata a sinistra esce dal blocco e non viene eseguita

- le parentesi nell'ultima riga: main senza parentesi non fa niente, e non dà errore

Oggi si copia. Perché serva tutto questo giro si vede con le funzioni.

16. COME SI LEGGE UN PROGRAMMA SCRITTO DA ALTRI

Dire cosa stamperà, prima di eseguirlo

Riepilogo di una spedizione LogiSud

```
"""Riepilogo di una spedizione LogiSud."""

TARIFFA_BASE = 8.50

RIGA = "-----"

def main():

    """Stampa il riepilogo di un collo in partenza."""

    codice = "LS-2026-MI-0042"

    zona = "SUD"

    colli = 3

    print(RIGA)

    print("Codice:", codice, "| Zona:", zona)

    print("Colli:", colli, end=" ")

    print("| Tariffa base:", TARIFFA_BASE)

    print("Totale:", TARIFFA_BASE * colli)

    print(RIGA)

if __name__ == "__main__":

    main()
```

Non lanciatelo: leggetelo e dite riga per riga cosa comparirà a schermo.

L'output: cinque righe, non sei

```
-----  
Codice: LS-2026-MI-0042 | Zona: SUD  
Colli: 3 | Tariffa base: 8.5  
Totale: 25.5  
-----
```

```
Le tre domande da farsi sempre  
  A cosa serve? ..... la docstring in cima, se c'è  
  Da dove arrivano i dati? .. le costanti e le assegnazioni  
  Cosa produce? ..... i print, letti dall'alto in basso
```

Il terzo print finisce con end e la riga prosegue con il quarto: per questo le righe sono cinque e non sei. E 8.50 è diventato 8.5, perché a Python lo zero finale non interessa.

RIEPILOGO

Cosa portiamo a casa, e cosa si fa adesso

Riepilogo della giornata (1/2)

Argomento	Concetti chiave
Come funziona il corso	Undici giornate dentro un percorso di 248 ore: qui le fondamentali, poi l'AI applicata
Il computer esegue	CPU una istruzione alla volta, dall'alto in basso; la RAM si svuota alla chiusura
Linguaggio di programmazione	Sintassi (non parte) contro semantica (parte e sbaglia)
Compilato e interpretato	Python è interpretato: si lancia subito, l'errore salta fuori all'esecuzione
Perché Python	Si legge; c'è già tutto pronto (ma noi non usiamo niente); è lento
Ambiente	Python 3.12 e PyCharm (niente estensioni); Add to PATH; python --version (python3 su Mac)
Terminale	cd, dir/ls, TAB, freccia in su; si lancia dalla cartella del file
REPL	Palestra per dubbi da trenta secondi; mostra il valore anche senza print
Primo file	Creare, scrivere, salvare, eseguire; niente spazi né accenti; occhio a .py.txt
print()	Argomenti multipli con la virgola, sep=, end=, print() vuoto per la riga bianca

Riepilogo della giornata (2/2)

Argomento	Concetti chiave
Virgolette ed escape	Singole e doppie equivalenti; \n a capo, \t tabulazione, \\ backslash
Variabili	nome = valore; l'= attacca un'etichetta; snake_case; si usa dopo averla creata
Numeri	+ - * /; la divisione dà sempre un numero con la virgola; decimali col punto
Testo	Il + fra testi incolla senza spazi; "5" + "3" fa 53; le righe di separazione in una costante
Testo e numeri insieme	"Totale: " + 244.0 è TypeError; oggi la virgola, le f-string arrivano domani
Commenti	# fino a fine riga; si commenta il perché, non il cosa; docstring in cima
Costanti e PEP 8	Valori fissi in MAIUSCOLO; niente magic numbers; 4 spazi, righe entro 88 caratteri
Traceback	Si legge dal basso: ultima riga = cosa, riga File = dove
Struttura standard	Docstring, costanti, def main():, guard finale — formula da copiare, si spiega con le funzioni

Grazie.

shaping the skills of tomorrow

